

Virtual Memory

- **Virtual Memory** is a **memory management technique** that allows the execution of processes that may **not be completely loaded into physical (main) memory**.
- It gives an **illusion** to each process that it has **a large, continuous block of memory**, even though the actual physical memory (RAM) may be much smaller.

Objectives of Virtual Memory

- A program doesn't need to be fully loaded in memory to run. Only the needed parts are loaded.
- Programs can be bigger than the physical memory available in the system.
- Virtual memory creates the illusion of a large memory, even if the actual memory (RAM) is small.
- It uses both RAM and disk storage to manage memory, loading only parts of programs into RAM as needed.
- This allows the system to run more programs at once and manage memory more efficiently.

How It Works

Virtual memory uses a combination of:

1. **Paging (or segmentation)**
2. **Demand paging**
3. **Page replacement algorithms**

Step-by-Step Process:

1. Program divided into pages (logical units).
2. Physical memory divided into frames (same size).
3. Only some pages are loaded into RAM (those needed immediately).
4. When CPU references a page not in memory, → page fault occurs.

5. The OS:

- Finds a free frame (or replaces one),
- Loads the required page from disk into that frame,
- Updates the **page table**.

6. CPU resumes execution.

Types of Virtual Memory Implementation

1. **Demand Paging** → Pages loaded only when needed (most common).
2. **Demand Segmentation** → Segments loaded when required (used in segmented systems).

Advantages of Virtual Memory

- Efficient Memory Use
- Multiprogramming
- Large Address Space
- Isolation & Protection
- Fast Context Switching

Disadvantages

- Page Fault Overhead
- Thrashing
- More Hardware
- Disk Wear

Thrashing

- Thrashing occurs in a computer system when the CPU spends most of its time swapping pages in and out of memory rather than executing actual processes.
- This happens when the demand for memory exceeds the available physical memory, leading to excessive paging activity.
- Each process needs a certain number of **frames** (pages in memory) to execute efficiently.
- When the **degree of multiprogramming** (number of processes in memory) increases beyond a limit, there aren't enough frames for all processes.
- As a result, pages are **continuously swapped in and out** — this excessive page faulting is called **thrashing**.

Causes of Thrashing

1. **High degree of multiprogramming**
→ Too many processes compete for limited memory.
2. **Insufficient physical memory**
→ Not enough frames to hold the active working sets of all processes.
3. **Poor page replacement policy**
→ Frequently replaces pages that are still needed.
4. **Improper allocation of frames**
→ Some processes may get too few frames.

Working Set Model (to avoid thrashing)

- The **Working Set Model** defines the set of pages that a process is currently using.
- The **working set window (Δ)** determines how many recent page references are considered.
- If the total demand for frames (sum of all working sets) exceeds available frames, **thrashing** will occur.

Effects of Thrashing

Aspect	Effect
CPU Utilization	Decreases sharply because CPU waits for I/O (paging).
Response Time	Increases drastically.
Throughput	Decreases — fewer processes get completed.
System Performance	Overall degraded — system appears “hung” or very slow.

Prevention / Control Methods

1. Working Set Model

→ Ensure enough frames are allocated to cover each process's working set.

2. Page Fault Frequency (PFF) Control

→ Monitor page fault rate;

- If too high → allocate more frames.
- If too low → reduce frames or increase multiprogramming.

3. Reduce Multiprogramming

→ Suspend or swap out some processes.

4. Use better page replacement algorithms

→ Like LRU (Least Recently Used) or MFU (Most Frequently Used).

5. Increase Physical Memory

→ Adds more frames, reducing competition.

Example

Suppose 4 processes are running and total 100 frames are available. Each process ideally needs 30 frames (total 120 required). The OS will continuously swap pages because not all processes have enough frames → **thrashing occurs**.

SEGMENTATION

- **Segmentation is a memory management technique where a process is divided into variable-sized chunks called segments.**
- Unlike paging, segmentation matches the user's logical view of a program (functions, arrays, modules) to physical memory.
- It reflects the user's view of memory rather than the computer's physical organization, making it easier to manage and protect processes.
- Each segment represents a **logical part** of the program, such as:
 - Code (instructions)
 - Data (variables)
 - Stack
 - Heap
 - Functions, arrays, objects, etc.

Key Features of Segmentation

1. **Variable-sized divisions:** Segments can have different lengths, depending on the program's requirements.
2. **Logical division of memory:** Segments represent meaningful units like code, stack, data, or modules.
3. **Two-part address:** A logical address consists of:
 - **Segment number (s):** Identifies which segment is being referred to.
 - **Offset (d):** Specifies the exact location within that segment or distance (in bytes) from the start of that segment.

Logical Address = ⟨Segment number, Offset⟩

4. **Protection and sharing:** Different segments can have access rights (read, write, execute) and can be shared among processes.
5. **No internal fragmentation:** Since segments are not fixed in size.
6. **External fragmentation:** Can occur when free memory is divided into small scattered blocks.

Structure of a Segment

Each process is divided into *segments*, numbered from 0, 1, 2, ...

Segment No.	Segment Name	Base Address	Length (Size)
0	Code	1400	400
1	Data	1800	300
2	Stack	2100	200

Segment Table

- A table stores the information about all such segments and is called Segment Table.
- It maps a two-dimensional Logical address into a one-dimensional Physical address. It's each table entry has:
 - **Base Address:** It contains the starting physical address where the segments reside in memory.
 - **Segment Limit:** Also known as segment offset. It specifies the length of the segment.

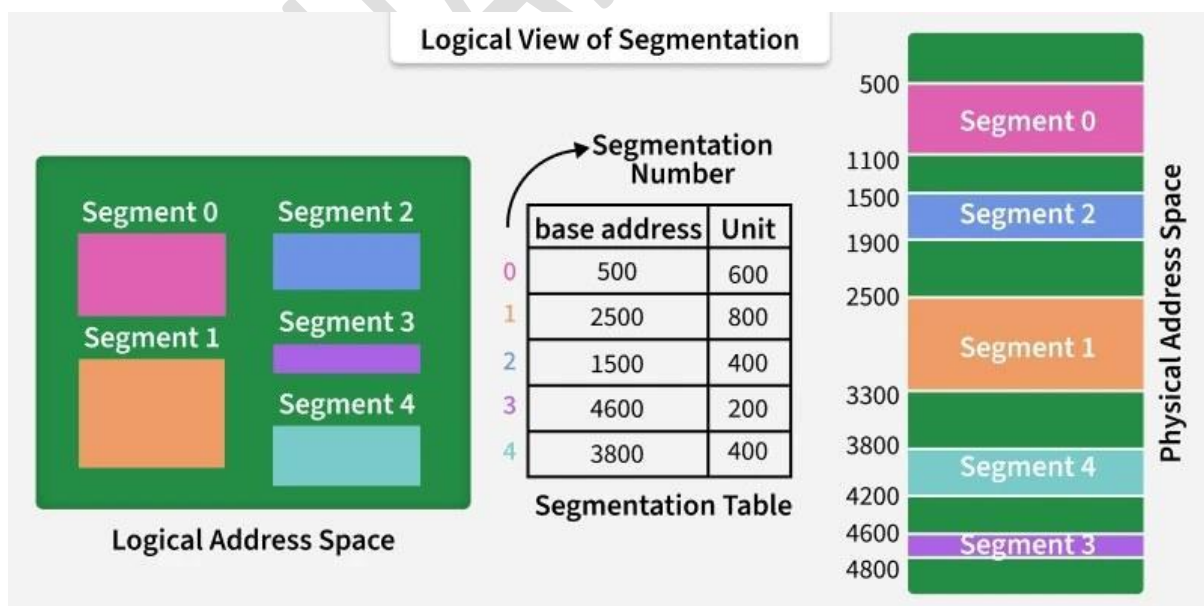


Fig.: Segmentation

Translation of Two-dimensional Logical Address to Dimensional Physical Address.

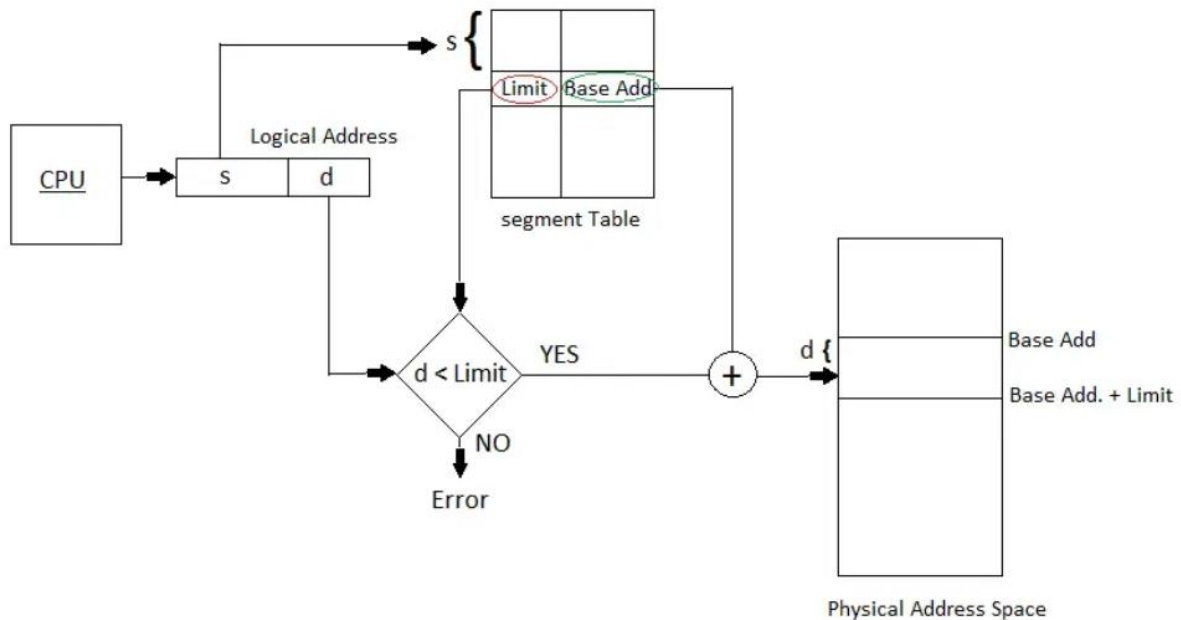


Fig.: Translation

Address Translation in Segmentation

The address generated by the CPU is divided into:

- **Segment number (s):** Number of bits required to represent the segment.
- **Segment offset (d):** Number of bits required to represent the position of data within a segment.

When CPU generates a logical address = $\langle s, d \rangle$:

1. The **segment number (s)** is used to index the **segment table**.
2. Check if **d < limit** (to ensure the offset is valid).
 - If not $\rightarrow$ **trap: segmentation fault**.
3. **Physical Address (PA)** = Base + Offset = Base(s) + d

Advantages of Segmentation in Operating System

- **Reduced Internal Fragmentation:** Segments are sized as per program needs, minimizing wasted space.
- **Smaller Segment Table:** Requires less space compared to page tables.
- **Better CPU Utilization:** Entire modules are loaded at once, improving performance.
- **Closer to User's View:** Programs can be divided into logical modules, matching how users think.
- **User-Controlled Size:** Segment size is defined by the user, unlike fixed page size in paging.
- **Security & Separation:** Segments help isolate sensitive data and operations.

Disadvantages of Segmentation in Operating System

- **External Fragmentation:** Free memory gets scattered, leading to wasted space.
- **Overhead:** Maintaining segment tables requires extra memory and management.
- **Slower Access:** Two memory lookups (segment table + main memory) increase access time.
- **Complexity:** Managing multiple variable-sized segments is harder than paging.
- **Segmentation Faults:** Errors may occur if a program tries to access memory outside its segment.

Formula Reminder

For segmentation:

$$\text{Physical Address (PA)} = \text{Base (from segment table)} + \text{Offset}$$

But only if:

$$\text{Offset} < \text{Limit}$$

Otherwise → **Segmentation fault (invalid address)**

NUMERICAL QUESTIONS

Q1. A process has the following segment table:

Segment	Base	Limit
0	1000	400
1	2000	600
2	3000	500

Find the **physical address** for the following logical addresses:

1. $\langle 0, 200 \rangle$
2. $\langle 1, 550 \rangle$
3. $\langle 2, 600 \rangle$

Solution:

1. $\langle 0, 200 \rangle \rightarrow$ Base = 1000, Limit = 400
 $\rightarrow 200 < 400$
 $\rightarrow \mathbf{PA = 1000 + 200 = 1200}$
2. $\langle 1, 550 \rangle \rightarrow$ Base = 2000, Limit = 600
 $\rightarrow 550 < 600$
 $\rightarrow \mathbf{PA = 2000 + 550 = 2550}$
3. $\langle 2, 600 \rangle \rightarrow$ Base = 3000, Limit = 500
 $\rightarrow 600 > 500$
 $\rightarrow \mathbf{Segmentation\ fault}$

Q2. A CPU generates a logical address of 32 bits, where:

- 12 bits are used for **offset**
- 4 bits for **segment number**

Find:

1. Maximum number of segments

2. Maximum segment size
3. If segment 5 has base = 4000 and limit = 2000, find the physical address for logical address <5, 1500>.

Solution:

1. Segment number bits = 4
→ $2^4 = 16$ segments
2. Offset bits = 12
→ $2^{12} = 4096$ bytes (max segment size)
3. Given <5, 1500>
→ Base = 4000, Limit = 2000 → Offset 1500 < 2000
→ **PA = 4000 + 1500 = 5500**

Q3. Given:

Segment	Base	Limit
0	1200	500
1	2000	800
2	3400	700
3	4100	1000

Find physical addresses for:

1. <0, 450>
2. <1, 900>
3. <3, 999>

Demand Paging

- **Demand Paging** is a memory management technique in which **pages are loaded into main memory only when they are needed** (on demand), **not in advance**.
- It is a type of **lazy loading** — the OS delays loading of a page until it is actually referenced during program execution.

Why Do We Need Demand Paging?

In **pure paging**, all pages of a process are loaded into memory before execution starts. This wastes memory if some pages are **never used**.

To overcome this, **demand paging** loads **only required pages**, reducing:

- Memory usage
- I/O time
- Process startup time

Basic Working of Demand Paging

When a process starts:

- Only a **small portion** of its pages (often none) are loaded in RAM.
- The rest are kept on **secondary storage (disk)**.

Steps in Execution:

1. CPU generates a **logical address**.
2. MMU (Memory Management Unit) checks the page table entry.
 - If the page is in memory → Access directly.
 - If not in memory → triggers a Page Fault.

Page Fault

A **page fault** occurs when a process tries to access a page **not currently loaded** in physical memory.

Steps to Handle a Page Fault (Important)

1. **CPU traps to OS** (page fault interrupt).
2. OS checks:
 - Is the reference valid (part of the process)?
 - If invalid → terminate process (segmentation fault).
3. If valid:
 - Locate the page on the **disk** (swap area).
4. **Find a free frame** in main memory:
 - If available, use it.
 - If not, use a **page replacement algorithm** (e.g., FIFO, LRU, Optimal) to evict a page.
5. **Read the page** from disk into the free frame.
6. **Update the page table** (mark “valid bit = 1”).
7. **Restart the instruction** that caused the fault.

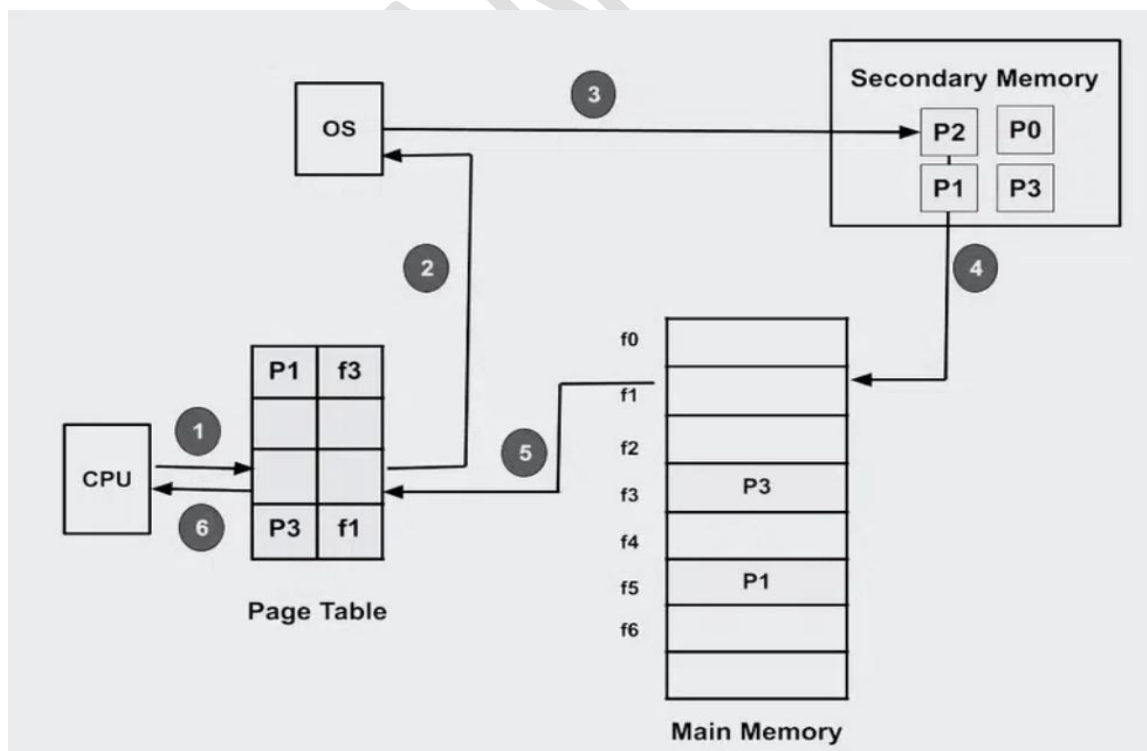


Fig: Demand Paging

Example

Suppose a process has 5 pages (P0–P4), and only P0 is initially loaded.

If the process references P3 →
→ Page fault occurs → OS loads P3 from disk to RAM.
Next time, if P3 is referenced again → no page fault (already in memory).

Page Table Entry (PTE) Structure in Demand Paging

Field	Description
Frame Number	Indicates where the page is loaded (if valid).
Valid/Invalid Bit	1 → Page in memory; 0 → Page not in memory.
Modified Bit (Dirty Bit)	1 → Page changed since loaded; must be written back to disk.
Reference Bit	Used by replacement algorithms.

How Demand Paging in OS Affects System Performance?

Demand paging can improve system performance by reducing the memory needed for programs and allowing multiple programs to run simultaneously.

- If not implemented properly, it can cause performance issues.
- When a program needs a part that isn't in the main memory, the operating system must fetch it from the hard disk, which takes time and pauses the program.
- This can cause delays and if the system runs out of memory, it will need to frequently swap pages in and out, increasing delays and reducing performance.

Effective Access Time (EAT)-Important

When demand paging is used, memory access time is not constant due to page faults.

Let:

- **p** = page fault rate ($0 \leq p \leq 1$)
- **ma** = memory access time (say 100 ns)
- **page fault service time** = (disk access + swap + table update), say 10 ms
= 10,000,000 ns

Then,

$$\text{EAT} = (1 - p) * \text{ma} + p * \text{page fault time}$$

Example:

If **ma** = 100 ns, **page fault time** = 10,000,000 ns, **p** = 0.001 (0.1%)

Then,

$$\begin{aligned}\text{EAT} &= (1 - 0.001) * 100 + 0.001 * 10,000,000 \\ &= 100 + 10,000 = 10,100\end{aligned}$$

Even small page fault rates can drastically increase access time.

Advantages

1. Efficient memory use
2. Faster program startup
3. More processes can fit in RAM, leading to higher multiprogramming.

Disadvantages

1. Page fault overhead
2. Thrashing
3. More hardware complexity

Page Replacement Algorithms

- In an operating system that uses paging, a page replacement algorithm is needed when a page fault occurs and no free page frame is available.
- In this case, one of the existing pages in memory must be replaced with the new page.

The virtual memory manager performs this by:

1. Selecting a victim page using a page replacement algorithm.
2. Marking its page table entry as “not present.”
3. If the page was modified (dirty), write it back to disk before replacement.

The efficiency of a page replacement algorithm directly affects the page fault rate, which in turn impacts system performance.

Common Page Replacement Techniques

- First In First Out (FIFO)
- Optimal Page replacement
- Least Recently Used (LRU)
- Most Recently Used (MRU)

1. First In First Out (FIFO)

- This is the simplest page replacement algorithm.
- In this algorithm, the operating system keeps track of all pages in the memory in a queue, the oldest page is in the front of the queue.
- When a page needs to be replaced page in the front of the queue is selected for removal.

Example 1: Consider page reference string 1, 3, 0, 3, 5, 6, 3 with 3-page frames. Find the number of page faults using FIFO Page Replacement Algorithm.

Page
reference

1, 3, 0, 3, 5, 6, 3

1	3	0	3	5	6	3
		0	0	0	0	3
	3	3	3	3	6	6
1	1	1	1	5	5	5
Miss	Miss	Miss	Hit	Miss	Miss	Miss

Total Page Fault = 6

FIFO - Page Replacement

- Initially, all slots are empty, so when 1, 3, 0 came they are allocated to the empty slots ---> **3 Page Faults.**
- When 3 comes, it is already in memory so ---> **0 Page Faults.**
- Then 5 comes, it is not available in memory, so it replaces the oldest page slot i.e 1. ---> **1 Page Fault.**
- 6 comes, it is also not available in memory, so it replaces the oldest page slot i.e 3 ---> **1 Page Fault.**
- Finally, when 3 come it is not available, so it replaces 0 1-page **fault.**

Belady's Anomaly

In FIFO, **increasing the number of frames** can sometimes **increase** page faults

Example reference string: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5

- 3 frames → 9 faults
- 4 frames → 10 faults

2. Optimal Page Replacement

In this algorithm, pages are replaced which would not be used for the longest duration of time in the future.

Example: Consider the page references 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 3 with 4-page frame. Find number of page fault using Optimal Page Replacement Algorithm.

Page
reference

7,0,1,2,0,3,0,4,2,3,0,3,2,3

No. of Page frame - 4

7	0	1	2	0	3	0	4	2	3	0	3	2	3
			2	2	2	2	2	2	2	2	2	2	2
		1	1	1	1	1	4	4	4	4	4	4	4
	0	0	0	0	0	0	0	0	0	0	0	0	0
7	7	7	7	7	3	3	3	3	3	3	3	3	3
Miss	Miss	Miss	Miss	Hit	Miss	Hit	Miss	Hit	Hit	Hit	Hit	Hit	Hit

Total Page Fault = 6

Optimal Page Replacement

- Initially, all slots are empty, so when 7 0 1 2 are allocated to the empty slots ---> 4 Page faults
- 0 is already there so ---> 0 Page fault. when 3 came it will take the place of 7 because it is not used for the longest duration of time in the future---> 1 Page fault.
- 0 is already there so ---> 0 Page fault. 4 will takes place of 1 ---> 1 Page Fault.

Now for the further page reference string ---> 0 Page fault because they are already available in the memory.

Optimal page replacement is perfect, but not possible in practice as the operating system cannot know future requests.

3. Least Recently Used

In this algorithm, page will be replaced which is least recently used.

Example Consider the page reference string 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 3 with 4-page frames. Find number of page faults using LRU Page Replacement Algorithm.

Page
reference

7,0,1,2,0,3,0,4,2,3,0,3,2,3

No. of Page frame - 4

7	0	1	2	0	3	0	4	2	3	0	3	2	3
			2	2	2	2	2	2	2	2	2	2	2
		1	1	1	1	1	4	4	4	4	4	4	4
	0	0	0	0	0	0	0	0	0	0	0	0	0
7	7	7	7	7	3	3	3	3	3	3	3	3	3
Miss	Miss	Miss	Miss	Hit	Miss	Hit	Miss	Hit	Hit	Hit	Hit	Hit	Hit

Total Page Fault = 6

Here LRU has same number of page fault as optimal but it may differ according to question.

Least Recently Used - Page Replacement

- Initially, all slots are empty, so when 7 0 1 2 are allocated to the empty slots ---> 4 Page faults.
- 0 is already there so ---> 0 Page fault. when 3 came it will take the place of 7 because it is least recently used ---> 1 Page fault.
- 0 is already in memory so ---> 0 Page fault.
- 4 will takes place of 1 ---> 1 Page Fault.
- Now for the further page reference string ---> 0 Page fault because they are already available in the memory.

4. Most Recently Used (MRU)

- In this algorithm, page will be replaced which has been used recently.
- Belady's anomaly can occur in this algorithm.

Example 4: Consider the page reference string 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 3 with 4-page frames. Find number of page faults using MRU Page Replacement Algorithm.

Page
reference

7,0,1,2,0,3,0,4,2,3,0,3,2,3

No. of Page frame - 4

7	0	1	2	0	3	0	4	2	3	0	3	2	3
			2	2	2	2	2	2	3	0	3	2	3
		1	1	1	1	1	1	1	1	1	1	1	1
	0	0	0	0	3	0	4	4	4	4	4	4	4
7	7	7	7	7	7	7	7	7	7	7	7	7	7
Miss	Miss	Miss	Miss	Hit	Miss	Miss	Miss	Hit	Miss	Miss	Miss	Miss	Miss

Total Page Fault = 12

Most Recently Used - Page Replacement

- Initially, all slots are empty, so when 7 0 1 2 are allocated to the empty slots ---> 4 Page faults
- 0 is already there so---> no page fault
- when 3 comes it will take place of 0 because it is most recently used ---> 1 Page fault
- when 0 comes it will take place of 3 ---> 1 Page fault
- when 4 comes it will take place of 0 ---> 1 Page fault
- 2 is already in memory so ---> 0 Page fault
- when 3 comes it will take place of 2 ---> 1 Page fault
- when 0 comes it will take place of 3 ---> 1 Page fault
- when 3 comes it will take place of 0 ---> 1 Page fault
- when 2 comes it will take place of 3 ---> 1 Page fault
- when 3 comes it will take place of 2 ---> 1 Page fault